

A project on

MathMitra

Submitted in partial fulfillment of the requirements of

Field Project (NARFPL41)

Bachelors of Automation & Robotics Engineering

by

Sakshi Gupta (20)

Khushi Sayani (50)

Nandini Gupta (19)

Riya Balani (06)

Under the guidance of Prof Dr. (Mrs.) Sangeetha P Ram



Department of Automation & Robotics Engineering

Vivekanand Education Society's Institute of Technology

Academic Year: 2025-2026

CERTIFICATE

This is to certify that the project entitled “**MathMitra**” is a bonafide work of 1.Sakshi Gupta , 2.Khushi Sayani, 3.Nandini Gupta , 4.Riya balani Submitted to **Vivekanand Education Society Institute Of Technology** in partial fulfillment of the requirement for the award of the degree of “**Undergraduate**” in “**Automation & Robotics Engineering**”

Prof.

Project Guide

Dr. (Mrs.) Sangeetha P Ram

HoD, AURO

Project Report Approval

This project report entitled “MathMitra” by

1. Sakshi Gupta (20)
2. Khushi Sayani (50)
3. Nandini Gupta (19)
4. Riya Balani (06)

submitted to **Vivekanand Education Society Institute Of Technology** in partial fulfillment of the requirement for the award of the degree of **Bachelors of Engineering in Automation & Robotics**.

Examiners:

1. Mr. Kader Shaikh
2. Dr. Nilima Warke

Date: 18-04-2026

Place: VESIT, Mumbai

INDEX

SR NO.	CONTENT NAME	PAGE NO.
1	Project abstract	
2	Project objectives	
3	Proof of field interactions (Geo Tagged photos/videos of Field Interaction, appreciation letter, etc)	
4	Introduction	
5	Block diagram & operation of the system	
6	Component list and specification	
7	Connection/signal condition diagram description	
8	Program and flow chart description	
9	Advantages and Disadvantage	
10	Conclusion	
11	Future work	
12	References	
13	Log book	

Project Abstract

MathMitra is a comprehensive, technology-driven educational platform designed to transform the way students learn mathematics by making it interactive, engaging, and accessible for all learners, including those with disabilities. The project aims to bridge the gap between traditional teaching methods and modern digital learning by integrating gamification, adaptive learning, accessibility tools, and hardware-based interaction into a single unified system.

The platform offers a structured practice environment where students can solve mathematical problems based on their grade level and specific topics such as counting, addition, subtraction, multiplication, fractions, and algebra. The system dynamically adjusts the difficulty level of questions based on the student's performance using accuracy-based evaluation. It also records user attempts, analyzes mistakes, and provides personalized feedback to improve conceptual understanding.

To enhance engagement, MathMitra incorporates multiple interactive and animated games such as Number Builder, Equation Builder, Math Race, Borrowing Quest, and Space Math Adventure. These games are designed to promote active learning through visual representation, drag-and-drop interactions, real-time feedback, and reward systems like points, stars, and progress tracking. The use of animations and gamified elements helps maintain user interest while reinforcing mathematical concepts.

A major highlight of the project is its strong emphasis on accessibility, particularly for hearing-impaired students. The platform includes an Accessibility Module featuring a Sign Language Avatar that interprets textual content into sign language, enabling better understanding for deaf users. Additionally, visual aids, simplified text explanations, and interactive UI components ensure that the platform is inclusive and user-friendly. The project further extends accessibility by integrating a hardware-based gesture recognition system using a smart glove. This glove is equipped with flex sensors to detect finger movements and a gyroscope (MPU6050) to track hand orientation. The sensor data is processed using a microcontroller such as ESP32, which transmits the data to the web application via wireless communication. The system interprets these gestures into meaningful inputs, allowing users to control the platform, interact with games, or communicate through sign-based commands without relying on traditional input devices.

Overall, MathMitra represents an innovative fusion of education, gamification, accessibility, and hardware integration. It not only improves the effectiveness of math learning but also ensures inclusivity by supporting differently-abled users. The project demonstrates how emerging technologies can be leveraged to create a smart, interactive, and accessible learning ecosystem, making education more engaging and universally available.

Project Objectives

The primary objective of the **MathMitra** project is to develop an interactive, engaging, and accessible educational platform that enhances mathematics learning for students of different age groups and abilities. The project focuses on combining modern technologies with educational methodologies to create an inclusive and effective learning environment.

Key Objectives

1. Improve Mathematics Learning Experience

- To provide a structured platform for practicing mathematical concepts such as counting, arithmetic operations, fractions, and algebra.
- To simplify complex concepts using visual representations and interactive methods.

2. Implement Adaptive Learning

- To dynamically adjust the difficulty level of problems based on student performance.
- To track user progress and accuracy for personalized learning experiences.

3. Introduce Gamified Learning

- To develop interactive games (e.g., Number Builder, Equation Builder, Space Math Adventure) that make learning fun and engaging.
- To motivate students through rewards such as points, stars, and achievements.

4. Ensure Accessibility for All Users

- To design features specifically for hearing-impaired users, including sign language support and visual aids.
- To create a user-friendly interface that is easy to understand and navigate for all learners.

5. Integrate Gesture-Based Interaction

- To develop a hardware-based smart glove using flex sensors and a gyroscope for detecting hand gestures.
- To enable users to interact with the platform using gestures instead of traditional input devices.

6. Build a Scalable and Modern System

- To use modern technologies like React, Firebase, and microcontrollers for efficient performance.
- To ensure the platform is responsive and works across different devices.

7. Promote Inclusive and Innovative Education

- To bridge the gap between traditional teaching and modern digital learning.
- To create an educational system that supports both academic growth and accessibility needs.

Introduction

Mathematics is a fundamental subject that plays a crucial role in developing logical thinking and problem-solving skills. However, many students find mathematics difficult, uninteresting, and challenging to understand using traditional teaching methods. The lack of interactive learning tools and personalized guidance often leads to decreased motivation and poor conceptual clarity among learners.

With the advancement of technology, there is a growing need to transform conventional education systems into more engaging, adaptive, and student-friendly platforms. Digital learning solutions have the potential to make education more interactive through visual aids, real-time feedback, and gamified experiences. At the same time, it is equally important to ensure that such platforms are accessible to all students, including those with disabilities.

MathMitra is designed as an innovative solution to address these challenges. It is an interactive and accessible mathematics learning platform that combines adaptive practice, gamified learning, and assistive technologies to enhance the overall learning experience. The platform provides students with topic-based practice questions, performance tracking, and instant feedback to help them improve their understanding step by step. One of the key features of MathMitra is the integration of educational games that make learning enjoyable and engaging. Games such as Number Builder, Equation Builder, and Space Math Adventure encourage active participation and help students grasp mathematical concepts through visual and interactive methods rather than rote learning.

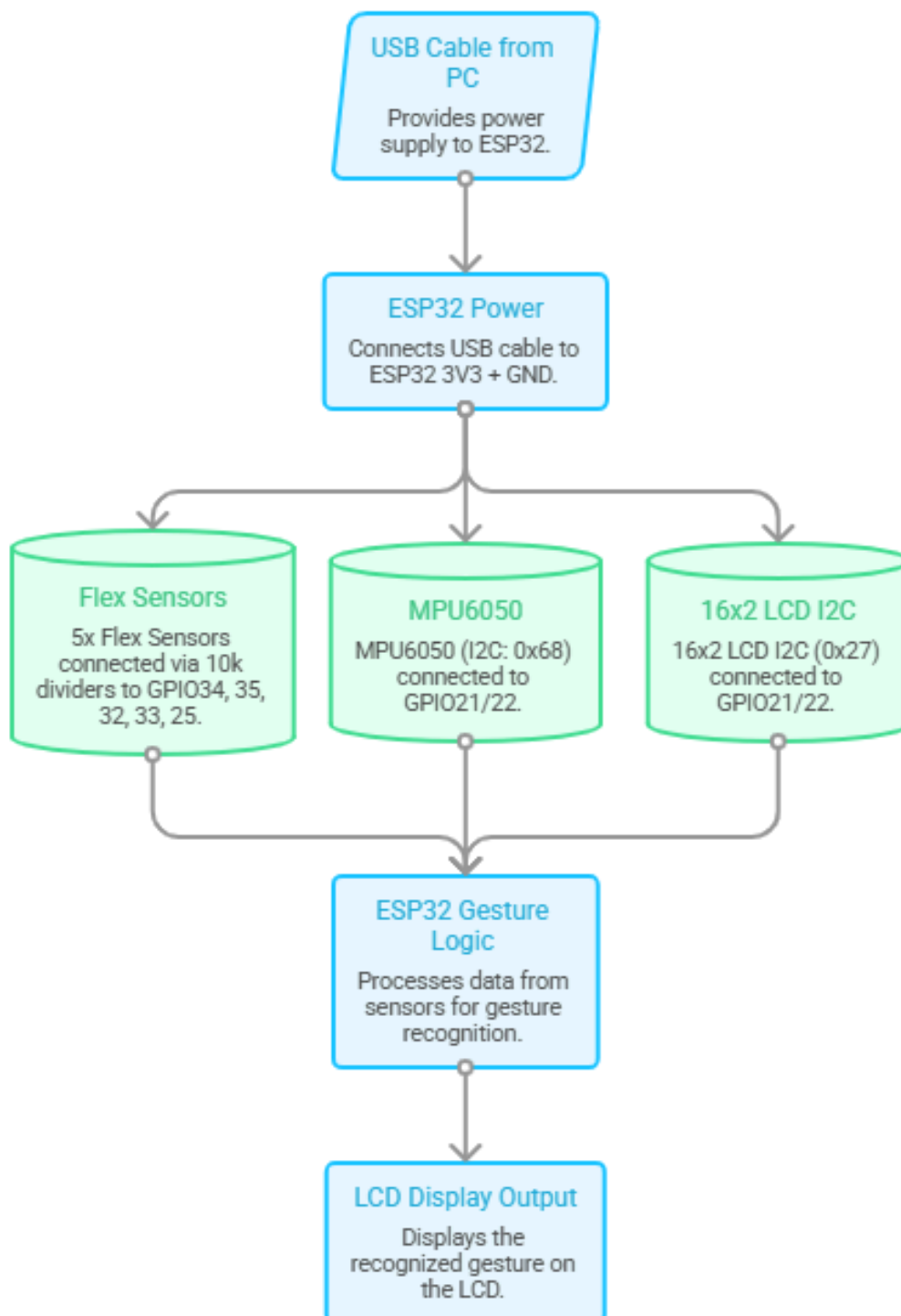
In addition to enhancing engagement, MathMitra places a strong emphasis on accessibility. The platform includes features such as sign language support, visual aids, and simplified content to assist hearing-impaired students. Furthermore, the project incorporates a hardware-based gesture recognition system using a smart glove equipped with flex sensors and a gyroscope. This allows users to interact with the system using hand gestures, making the platform more inclusive and innovative.

The system is built using modern technologies such as React for the frontend, Firebase for authentication and data management, and microcontrollers for hardware integration. These technologies ensure that the platform is scalable, responsive, and capable of delivering a smooth user experience.

In summary, MathMitra aims to revolutionize mathematics education by combining interactive learning, gamification, accessibility, and hardware-based innovation into a single platform. It not only improves the effectiveness of learning but also ensures that education is accessible and enjoyable for every student.

Block Diagram:

ESP32 Gesture Recognition System



Operation of the System:

1. Flex Sensor Operation

A flex sensor is a resistive strip whose resistance ranges from ~10 kΩ (flat) to ~30–50 kΩ (fully bent). Placed in a voltage-divider circuit, this resistance change yields a proportional voltage read by the ESP32 as a 12-bit ADC value (0–4095).

- Higher ADC value → finger more straight (lower resistance).
- Lower ADC value → finger more bent (higher resistance).

2. Gesture Recognition – Threshold-Based

Each ASL gesture is defined by five ADC threshold ranges (one per finger) plus wrist pitch and roll from the MPU6050. All seven values must fall within stored ranges for a gesture match. Example thresholds for ASL 'A':

Finger	Min ADC	Max ADC	Meaning
Thumb	800	1500	Bent/tucked (fist)
Index	700	1400	Bent
Middle	700	1400	Bent
Ring	700	1400	Bent
Pinky	700	1400	Bent

3. MPU6050 Wrist Orientation Enhancement

Several ASL handshapes share identical finger configurations but differ in wrist orientation. The MPU6050 accelerometer provides gravity-relative pitch and roll angles as additional matching conditions, reducing false positives.

- Pitch – detects whether the hand points up, forward, or down.
- Roll – detects whether the palm faces left or right.
- Example: ISL '1' (index up) vs. ISL 'L' (index sideways) – same fingers, different wrist angle.

4. ISL Gesture-to-Output Mapping

Sign	Finger State	Wrist Condition	LCD Output
A	All fingers bent, thumb tucked alongside fist	Neutral upright	ISL: A
B	Index, middle, ring, pinky straight; thumb bent inward	Palm facing forward	ISL: B
C	All fingers half-bent forming a C curve	Neutral	ISL: C
1	Index finger straight up, all others bent	Hand pointing upward	ISL: 1
2	Index and middle straight, others bent	Hand pointing upward	ISL: 2
Open Hand	All five fingers fully extended flat	Any	READY

5. Step-by-Step Logic

1. System boots. Initialise I²C bus on GPIO21 (SDA) and GPIO22 (SCL).
2. Initialise LCD at address 0x27. Enable backlight and display startup message.
3. Initialise MPU6050 at address 0x68. Compute zero-point offsets with glove held flat.
4. Read all 5 ADC pins. Average 5 samples per pin to reduce noise.
5. Read MPU6050. Compute pitch (rotation about X-axis) and roll (rotation about Y-axis).
6. Compare all 7 values (5 flex + pitch + roll) against every gesture in the library.
7. Match found → display the corresponding ISL character on LCD row 2.
8. No match → display 'Unknown' on LCD row 2.
9. Wait 500 ms. Return to Step 4 and repeat.

Algorithm and System Flow:

Startup Sequence

1. Initialize I2C bus (GPIO21 SDA, GPIO22 SCL).
2. Initialize LCD at address 0x27 (try 0x3F if no display). Display welcome message.
3. Initialize MPU6050 at address 0x68. Verify connection by reading WHO_AM_I register (expected 0x68).
4. Prompt user for Calibration Phase 1: FLAT HAND. Read 50 samples per flex sensor, compute average → store as openRef[5].
5. Prompt user for Calibration Phase 2: CLOSED FIST. Read 50 samples per flex sensor, compute average → store as closedRef[5].
6. Enter main recognition loop.

Main Recognition Loop

7. Read raw ADC values for all 5 flex sensors (analogRead on each GPIO). Average over 10 rapid samples to reduce noise.
8. Normalise each reading: $\text{bendPct}[i] = \text{clamp}((\text{raw}[i] - \text{openRef}[i]) / (\text{closedRef}[i] - \text{openRef}[i]) \times 100, 0, 100)$.
9. Read MPU6050 accelerometer data over I2C (AcX, AcY, AcZ raw 16-bit values).
10. Compute pitch = $\text{atan2}(\text{AcY}, \text{AcZ}) \times 180/\pi$. Compute roll = $\text{atan2}(\text{AcX}, \text{AcZ}) \times 180/\pi$.
11. Pass (bendPct[0..4], pitch, roll) to classifyGesture() function.
12. classifyGesture() iterates through all 13 gesture rule definitions in priority order. Returns matching label string or empty string if no match.
13. If no match: if previous label was valid and less than 1.5 s old, retain it (prevents flicker). Otherwise display 'UNKNOWN' or clear.
14. Apply debounce: do not re-display same label unless it changes.
15. Loop delay: 100 ms.

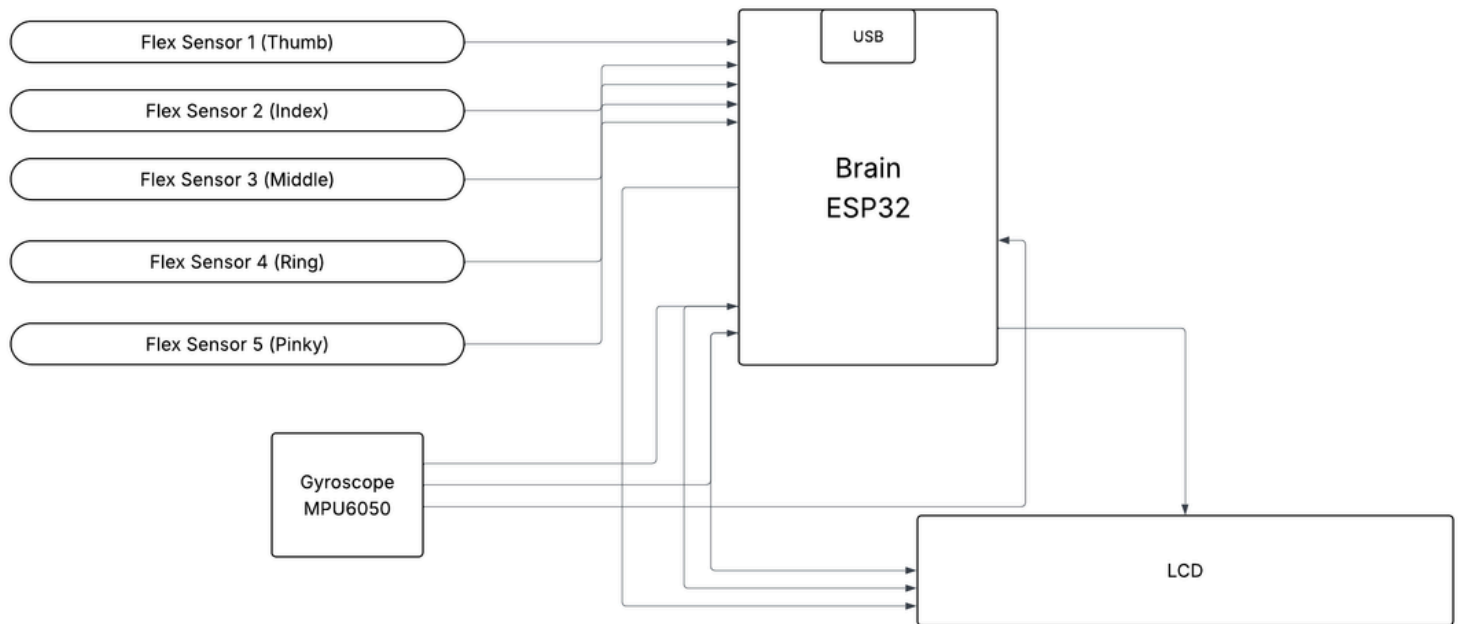
Classification Priority

Gestures with similar flex profiles are separated primarily by IMU orientation. The classification order is:

STOP (all bent + moderate positive roll)

16. Letter A (all bent + strong positive roll + thumb partially straight)
17. Letter B (all except thumb straight + slight positive pitch)
18. Letter C (all HB + strong negative roll / sideways palm)
19. Letter D (index straight + thumb HB + others bent + medium pitch)
20. Letter E (all bent + forward pitch + less roll than A)

Circuit diagram / signal condition diagram description:



The system follows a linear sense-process-display pipeline as described below:

SENSE → ACQUIRE → PROCESS → CLASSIFY → DISPLAY

(Flex Sensors + MPU6050) → (ESP32 ADC + I2C) → (Normalise + Fuse) → (Rule Engine) → (16×2 LCD)

Data Flow Description

1. Five flex sensors produce analog voltage signals proportional to finger bend angle. These are read by ESP32 GPIO ADC pins (12-bit, 0–4095 counts).
2. The MPU6050 provides 3-axis accelerometer and 3-axis gyroscope data over I2C. Pitch and roll angles are computed from accelerometer data using arctangent.
3. Both sensor streams are normalised against per-session calibration values (flat hand = 0%, fully bent = 100%).
4. A rule-based classifier compares normalised values against pre-defined gesture threshold windows.
5. The matched gesture label is transmitted to the 16×2 I2C LCD via I2C bus.

List of Components:

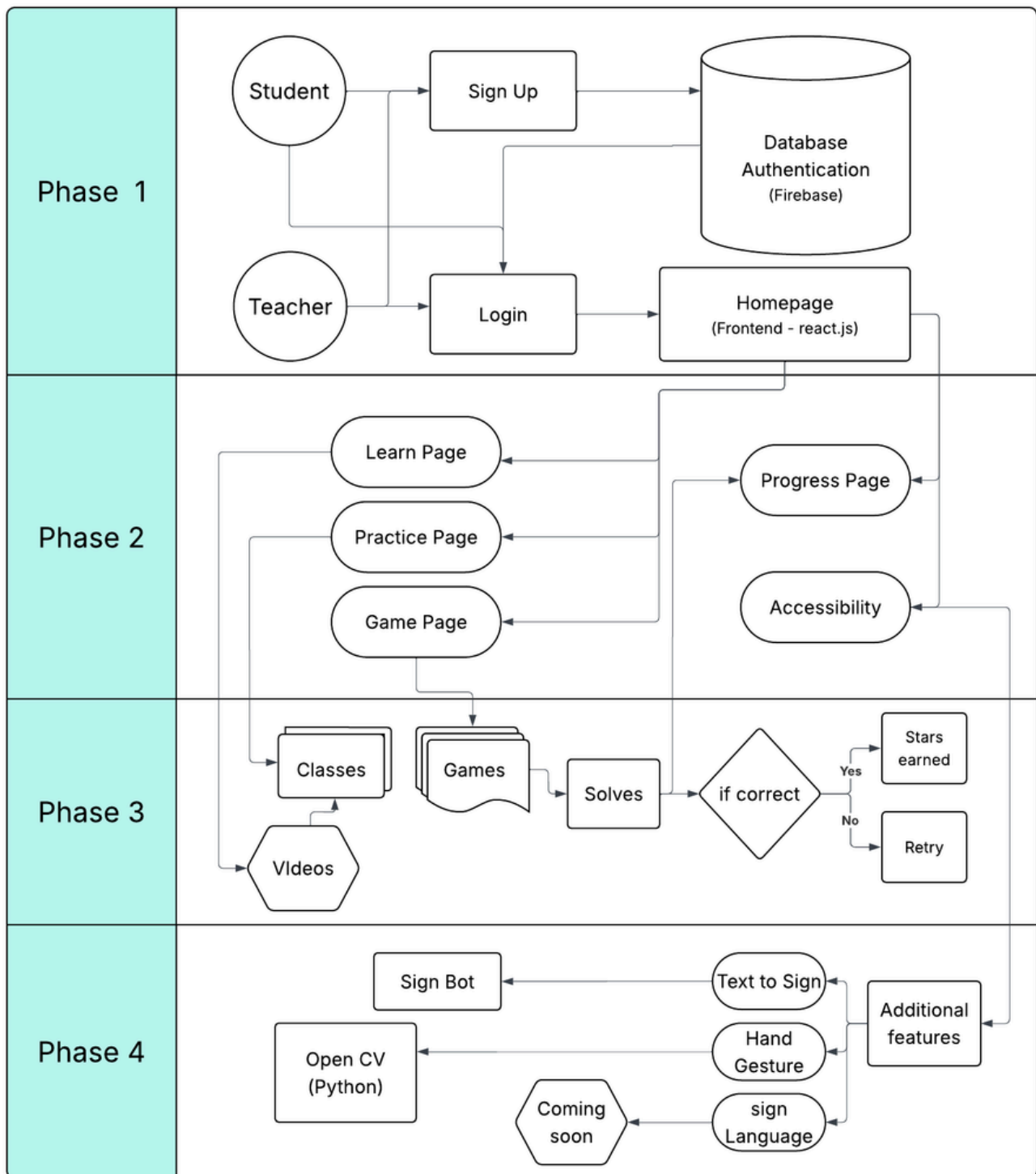
Hardware Components

1. **ESP32 Development Board**
 - Main controller (reads sensors, processes gestures, controls output)
2. **Flex Sensors (×5)**
 - One for each finger (thumb, index, middle, ring, pinky)
3. **MPU6050 Sensor Module**
 - 3-axis accelerometer + gyroscope (for pitch & roll detection)
4. **16×2 LCD Display (I2C module)**
 - Displays recognized ISL gestures
5. **Resistors (×5, typically 10 kΩ)**
 - Used in voltage divider circuits with flex sensors
6. **Breadboard**
 - For circuit prototyping and connections
7. **Jumper Wires**
 - For connecting components
8. **Glove (fabric base)**
 - To mount flex sensors on fingers
9. **USB Cable**
 - For programming and powering ESP32

Description of Components:

Component	Qty	Specification	Role
ESP32 Dev Board	1	Main microcontroller — dual-core, built-in ADC (12-bit), I2C, Wi-Fi (unused)	Central processing and sensor interfacing
MPU6050 (GY-521 module)	1	6-axis IMU — 3-axis accel + 3-axis gyro, I2C address 0x68	Wrist pitch and roll detection
Flex Sensor (2.2 inch)	5	Resistive strip — resistance increases from ~25 k Ω (flat) to ~90 k Ω (bent)	Detect bend angle of each finger
Resistor, 10 k Ω $\pm$ 5%	5	Fixed half of voltage divider for each flex sensor	Convert flex resistance to ADC-readable voltage
16×2 I2C LCD	1	LCD with I2C backpack, address 0x27 or 0x3F	Display recognised gesture labels
Breadboard (full/half size)	1	Solderless prototyping board	Mounting and wiring all components
Jumper Wires (M-M, M-F)	~30	Flexible connectors	Signal and power wiring
USB Micro-B Cable	1	Programming and serial monitoring	Flash firmware to ESP32

Flow Chart



Program code

[calibration code link:](#)

[calibration code.docx](#)

```
#define PIN_THUMB 33
#define PIN_INDEX 32
#define PIN_MIDDLE 35
#define PIN_RING 25
#define PIN_PINKY 34

#define CALIB_SAMPLES 50

int openVal[5];
int closeVal[5];

int readAvg(int pin, int samples){
    long sum = 0;
    for(int i = 0; i < samples; i++){
        sum += analogRead(pin);
        delay(5);
    }
    return (int)(sum / samples);
}

void setup(){
    Serial.begin(115200);
    Serial.println("=== FLEX SENSOR CALIBRATION ===");
    delay(3000);

    Serial.println(">> Keep hand OPEN (straight fingers)");
    Serial.println("Waiting 5 seconds...");
    delay(5000);

    openVal[0] = readAvg(PIN_THUMB, CALIB_SAMPLES);
    openVal[1] = readAvg(PIN_INDEX, CALIB_SAMPLES);
    openVal[2] = readAvg(PIN_MIDDLE, CALIB_SAMPLES);
    openVal[3] = readAvg(PIN_RING, CALIB_SAMPLES);
    openVal[4] = readAvg(PIN_PINKY, CALIB_SAMPLES);

    Serial.println("OPEN values captured!");
    delay(3000);

    Serial.println(">> Make a FIST (close all fingers)");
    Serial.println("Waiting 5 seconds...");
    delay(5000);

    closeVal[0] = readAvg(PIN_THUMB, CALIB_SAMPLES);
    closeVal[1] = readAvg(PIN_INDEX, CALIB_SAMPLES);
    closeVal[2] = readAvg(PIN_MIDDLE, CALIB_SAMPLES);
    closeVal[3] = readAvg(PIN_RING, CALIB_SAMPLES);
    closeVal[4] = readAvg(PIN_PINKY, CALIB_SAMPLES);
```

```

closeVal[2] = readAvg(PIN_MIDDLE, CALIB_SAMPLES);
closeVal[3] = readAvg(PIN_RING, CALIB_SAMPLES);
closeVal[4] = readAvg(PIN_PINKY, CALIB_SAMPLES);

Serial.println("CLOSED values captured!");

Serial.println("\n=== COPY THESE VALUES INTO gesture_glove.ino ===");

Serial.print("float openRef[5] = { ");
for(int i = 0; i < 5; i++){
    Serial.print(openVal[i]);
    if(i < 4) Serial.print(", ");
}
Serial.println(" };");

Serial.print("float closedRef[5] = { ");
for(int i = 0; i < 5; i++){
    Serial.print(closeVal[i]);
    if(i < 4) Serial.print(", ");
}
Serial.println(" };");

Serial.println("\n=== SANITY CHECK (delta should be > 100) ===");

const char* names[5] = {"Thumb","Index","Middle","Ring","Pinky"};
for(int i = 0; i < 5; i++){
    int delta = abs(closeVal[i] - openVal[i]);
    Serial.print(" ");
    Serial.print(names[i]);
    Serial.print(": open=");
    Serial.print(openVal[i]);
    Serial.print(" closed=");
    Serial.print(closeVal[i]);
    Serial.print(" delta=");
    Serial.print(delta);
    Serial.println(delta < 100 ? " ⚠️ LOW – check wiring" : " OK");
}
}

```

[final code link:](#)

[final code.docx](#)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <math.h>

#define FLEX_THUMB 34
#define FLEX_INDEX 35
#define FLEX_MIDDLE 32
#define FLEX_RING 33
#define FLEX_PINKY 25

const int FLEX_PINS[5] = { FLEX_THUMB, FLEX_INDEX, FLEX_MIDDLE, FLEX_RING, FLEX_PINKY };

#define I2C_SDA 21
#define I2C_SCL 22
#define MPU_ADDR 0x68
#define LCD_ADDR 0x27

#define CALIB_SAMPLES 50
#define NOISE_SAMPLES 10
#define LOOP_DELAY_MS 100
#define RETAIN_MS 1500

float openRef[5] = { 0 };
float closedRef[5] = { 0 };
float bendPct[5] = { 0 };

float pitch = 0.0f, roll = 0.0f;

String prevLabel = "";
unsigned long prevLabelTime = 0;
String displayedLabel = "";

LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);

bool mpuBegin() {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x6B);
    Wire.write(0x00);
    if (Wire.endTransmission(true) != 0) return false;
    return true;
}

bool mpuWhoAmI() {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x75);
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 1, true);
```

```

    if (Wire.available()) {
        uint8_t id = Wire.read();
        return (id == 0x68);
    }
    return false;
}

struct Accel {
    int16_t x, y, z;
};

Accel readAccel() {
    Wire.beginTransaction(MPU_ADDR);
    Wire.write(0x3B);
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 6, true);
    Accel a = { 0, 0, 0 };
    if (Wire.available() >= 6) {
        a.x = (Wire.read() << 8) | Wire.read();
        a.y = (Wire.read() << 8) | Wire.read();
        a.z = (Wire.read() << 8) | Wire.read();
    }
    return a;
}

float readFlexAvg(int pin, int samples) {
    long sum = 0;
    for (int i = 0; i < samples; i++) {
        sum += analogRead(pin);
        delay(2);
    }
    return (float)sum / samples;
}

float clampf(float v, float lo, float hi) {
    return v < lo ? lo : (v > hi ? hi : v);
}

void calibratePhase(const char* prompt, float* refArray) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Calib: ");
    lcd.print(prompt);
    lcd.setCursor(0, 1);
    lcd.print("Hold & press BTN");
    delay(4000);
    lcd.clear();
}

```

```

    lcd.setCursor(0, 0);
    lcd.print("Sampling...");

    for (int i = 0; i < 5; i++) {
        refArray[i] = readFlexAvg(FLEX_PINS[i], CALIB_SAMPLES);
    }

    lcd.setCursor(0, 1);
    lcd.print("Done!");
    delay(1000);
}

#define BENT(i) (bendPct[i] >= 65.0f)
#define HALF(i) (bendPct[i] >= 30.0f && bendPct[i] < 65.0f)
#define STRAIT(i) (bendPct[i] < 30.0f)

String classifyGesture() {
    if (BENT(0) && BENT(1) && BENT(2) && BENT(3) && BENT(4) && roll > 15.0f && roll < 60.0f) {
        return "STOP";
    }

    if (HALF(0) && BENT(1) && BENT(2) && BENT(3) && BENT(4) && roll > 50.0f) {
        return "Letter A";
    }

    if (BENT(0) && STRAIT(1) && STRAIT(2) && STRAIT(3) && STRAIT(4) && pitch > 10.0f && pitch < 45.0f) {
        return "Letter B";
    }

    if (HALF(0) && HALF(1) && HALF(2) && HALF(3) && HALF(4) && roll < -40.0f) {
        return "Letter C";
    }

    if (HALF(0) && STRAIT(1) && BENT(2) && BENT(3) && BENT(4) && pitch > 20.0f && pitch < 60.0f) {
        return "Letter D";
    }

    if (BENT(0) && BENT(1) && BENT(2) && BENT(3) && BENT(4) && pitch > 30.0f && roll < 40.0f) {
        return "Letter E";
    }

    return "";
}

void updateLCD(const String& label) {
    if (label == displayedLabel) return;
    displayedLabel = label;

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Gesture:");
    lcd.setCursor(0, 1);

```

```

if (label.length() == 0) {
    lcd.print("UNKNOWN ");
} else {
    lcd.print(label);
    for (int i = label.length(); i < 16; i++) lcd.print(' ');
}
}

void setup() {
    Serial.begin(115200);

    Wire.begin(I2C_SDA, I2C_SCL);

    lcd.init();
    lcd.backlight();
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Gesture Glove");
    lcd.setCursor(0, 1);
    lcd.print("Initialising...");
    delay(2000);

    if (!mpuBegin()) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("MPU6050 ERROR");
        lcd.setCursor(0, 1);
        lcd.print("Check wiring!");
        Serial.println("[ERROR] MPU6050 not responding on I2C.");
        while (true) delay(1000);
    }

    if (!mpuWhoAmI()) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("WHO AM I FAIL");
        lcd.setCursor(0, 1);
        lcd.print("Expected 0x68");
        Serial.println("[ERROR] MPU6050 WHO AM I mismatch.");
        while (true) delay(1000);
    }

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("MPU6050 OK");
    Serial.println("[OK] MPU6050 connected.");
    delay(1000);
}

```

```

    calibratePhase("FLAT HAND", openRef);

    Serial.print("[CAL] Open refs: ");
    for (int i = 0; i < 5; i++) {
        Serial.print(openRef[i]);
        Serial.print(' ');
    }
    Serial.println();

    calibratePhase("CLOSED FIST", closedRef);

    Serial.print("[CAL] Closed refs: ");
    for (int i = 0; i < 5; i++) {
        Serial.print(closedRef[i]);
        Serial.print(' ');
    }
    Serial.println();

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Ready!");
    delay(1000);
}

void loop() {
    for (int i = 0; i < 5; i++) {
        float raw = readFlexAvg(FLEX_PINS[i], NOISE_SAMPLES);

        float denom = closedRef[i] - openRef[i];
        float pct = 0.0f;
        if (fabsf(denom) > 1.0f) {
            pct = (raw - openRef[i]) / denom * 100.0f;
        }
        bendPct[i] = clampf(pct, 0.0f, 100.0f);
    }

    Accel a = readAccel();
    pitch = atan2f((float)a.y, (float)a.z) * 180.0f / M_PI;
    roll = atan2f((float)a.x, (float)a.z) * 180.0f / M_PI;

    String label = classifyGesture();

    unsigned long now = millis();
    if (label.length() > 0) {
        prevLabel = label;
        prevLabelTime = now;
    } else {
        if (prevLabel.length() > 0 && (now - prevLabelTime) < RETAIN_MS) {
            label = prevLabel;
        }
    }

```

I

```
updateLCD(label);  
Serial.printf("[BEND] T:%.1f I:%.1f M:%.1f R:%.1f P:%.1f | [IMU] pitch:%.1f roll:%.1f | [GES] %s\n",  
    bendPct[0], bendPct[1], bendPct[2], bendPct[3], bendPct[4],  
    pitch, roll, label.length() ? label.c_str() : "UNKNOWN");  
delay(LOOP_DELAY_MS);  
}
```

trial output:



Advantages and Disadvantages

Limitation

Single-Hand Constraint (Critical ISL Limitation)

Indian Sign Language, like most sign languages, uses both hands for a significant portion of its vocabulary. Many ISL letters (G, H, J, K, etc.) and most ISL words require coordinated two-hand gestures, directional movement, or facial expressions as grammatical markers. This system, being limited to a single glove, can only represent a subset of ISL — primarily static, single-hand handshapes. This limitation must be explicitly acknowledged in any demonstration: the system is a proof-of-concept and educational aid, not a complete ISL translator.

Static Gestures Only

The current classifier captures a single instantaneous reading (static pose). Dynamic ISL signs that involve movement trajectories (e.g., letters J, Z which require drawing in the air) cannot be detected with this approach. Dynamic gesture detection would require temporal sequence analysis (e.g., Hidden Markov Models or LSTM networks).

Limited Gesture Set

Only 13 gestures are implemented. Full ISL fingerspelling requires 26 alphabets plus digits and hundreds of word-level signs. This system covers approximately 23% of the ISL alphabet (A–F) and 50% of single-digit numbers.

Gesture Overlaps and Environmental Factors

Gestures B vs. Number 4 share nearly identical flex profiles; they are distinguished only by a narrow pitch window (10–15°). In practice, users may find this transition unreliable. Environmental vibration, arm movement, and sensor wear can shift ADC baselines between calibration and use.

Flex Sensor Aging

Flex sensors degrade over repeated bend cycles. After extended use, the sensor's resistance range may narrow, reducing the calibration separation between open and closed references. Sensors may need periodic replacement in a production system.

Conclusion

The Sign Language Recognition System using flex sensors and MPU6050 demonstrates an effective, low-cost solution for translating hand gestures into readable text. By combining flex sensors for finger movement and the MPU6050 for wrist orientation, the system improves gesture recognition accuracy, especially for visually similar signs. The ESP32 enables real-time processing, while a threshold-based algorithm ensures efficient classification. This system enhances communication accessibility for individuals with hearing and speech impairments and highlights the importance of calibration and sensor fusion. It also provides a scalable base for future improvements such as machine learning and mobile integration.

Alongside this, MathMitra is a comprehensive web-based educational platform designed to make mathematics learning interactive, engaging, and accessible. It integrates gamification, real-time gesture recognition, and inclusive design into a unified system.

The platform uses a modern full-stack architecture:

- **Frontend:** Built with React, TypeScript, Vite, and Tailwind CSS, featuring responsive design, smooth animations, and five interactive math games with progressive difficulty.
- **Backend:** Dual-server system with Node.js/Express (authentication, APIs, Firebase integration) and Python Flask (real-time gesture detection using OpenCV and MediaPipe).

The gesture recognition system processes live video streams, detects hand landmarks, and classifies gestures efficiently with optimized frame handling and thread-safe execution. The platform includes key features such as gamified learning (levels, scoring, feedback), accessibility options (gesture control, visual aids, text-to-sign), and secure user management via Firebase.

Major technical challenges such as video streaming latency, gesture detection performance, and cross-platform compatibility were successfully resolved through optimization techniques like frame polling, efficient encoding, and modular backend design.

The system is production-ready, with a clean and maintainable codebase, responsive UI, robust error handling, and scalable architecture. It supports multiple input methods and follows inclusive design principles to cater to diverse learners.

Overall, this project successfully combines embedded systems and modern web technologies to address real-world problems in accessibility and education. It demonstrates strong technical implementation, user-centric design, and scalability, making it a solid foundation for future enhancements such as AI-based personalization, mobile applications, expanded gesture libraries, and advanced learning analytics.

Future work

Future Improvements

Dual-Hand System

Adding a second glove with a separate ESP32 (or using a second I2C MPU6050 at address 0x69) would allow recognition of two-hand ISL signs, dramatically expanding the vocabulary coverage. Wireless coordination via ESP-NOW (no internet required) could synchronise readings between both hands.

Machine Learning Classification

Replacing the rule-based classifier with a k-NN or small neural network model (trained on recorded gesture samples) would improve accuracy, especially for ambiguous gestures like B/4 and A/E. TensorFlow Lite Micro can run inference on an ESP32, enabling on-device ML without cloud dependency.

Dynamic Gesture Recognition

Implementing a circular sensor buffer and a sliding-window gesture recogniser using the gyroscope data from MPU6050 (currently unused) would enable detection of movement-based ISL signs such as J, Z, and directional words.

Bluetooth / BLE Output

The ESP32's built-in BLE can transmit recognised gesture text to a smartphone app, enabling more flexible and accessible output. A companion Android/iOS app could provide voice output via text-to-speech, turning the glove into a full speech-to-text surrogate.

Haptic Feedback

A small vibration motor (ERM) connected to a GPIO pin with a transistor driver could provide haptic confirmation when a gesture is successfully recognised, giving the user immediate non-visual feedback without looking at the LCD.

Expanded ISL Vocabulary

Using a microSD card module (SPI) and storing gesture definitions as a JSON file would allow updating the gesture set without re-flashing firmware, enabling non-technical educators to customise vocabulary for specific communication needs.

References

1. React Documentation
<https://react.dev/>
(Used for building the frontend user interface and components)
 2. Firebase Documentation
<https://firebase.google.com/docs>
(Used for authentication and real-time database management)
 3. Tailwind CSS Documentation
<https://tailwindcss.com/docs>
(Used for designing responsive and modern UI)
 4. Framer Motion Documentation
<https://www.framer.com/motion/>
(Used for animations and interactive UI elements)
 5. MediaPipe Documentation
<https://developers.google.com/mediapipe>
(Used for hand gesture recognition and tracking)
 6. OpenCV Documentation
<https://docs.opencv.org/>
(Used for image processing and gesture detection)
 7. MPU6050 Sensor Guide
<https://www.invensense.com/products/motion-tracking/6-axis/mpu-6050/>
(Used for understanding gyroscope and accelerometer functionality)
 8. ESP32 Documentation
<https://docs.espressif.com/projects/esp-idf/en/latest/>
(Used for microcontroller programming and communication)
 9. Web Speech API
https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
(Used for speech synthesis features)
 10. Sign Language Research Resources
<https://www.w3.org/WAI/>
(Guidelines for accessibility and inclusive design)
-
1. Human-Computer Interaction Concepts
<https://www.interaction-design.org/>
 2. Gamification in Education Research Papers
<https://scholar.google.com/>
 3. IEEE Papers on Gesture Recognition
<https://ieeexplore.ieee.org/>

Log book - page 1

Sr. No	Date & Time	Action plan/ Event/ Operation	Result	Status
1	19/1/2026	Letterhead	Prepared and finalized the project letterhead for documentation.	completed
2	29/1/2026	Field visit	Visited the project site/company and collected relevant information for the project.	completed
3	9/2/2026	Problem Statement	Identified and defined the main problem statement of the project.	completed
4	12/2/20	NOC	Discussed the problem statement with the guide and made necessary modifications.	completed
5	23/2/2026	Frontend	Designed and developed the basic user interface for the project.	completed
6	5/3/2026	Extra Features	Added additional features to improve usability and functionality.	completed
7	10/3/2026	Backend	Started backend development and integrated database functionality	completed
8	13/3/2026	Guidance	Received guidance and feedback from the project mentor.	completed

Sr. No	Date & Time	Action plan/ Event / Operation	Result	Status
9	19/3/2026	Hardware Implementation	Mentor suggested hardware ideas	completed
10	24/3/2026	Git Hub Push	Push the project to github	completed
11	26/3/2026	Gameifications	Added various interactive Web games	completed
12	30/3/2026	Sign Language bot	Discussed the problem statement with the guide and made necessary modifications.	completed
13	3/4/2026	Front end	Implement alphabets ISL	completed
14	7/4/2026	Hand Gestures	Open CV and Media-pipe suggested by Mentor	completed
15	10/4/2026	Hardware – Hand sign language gesture	Simulations and connections are done on Wokwi	completed
16	15/4/2026	Comp	Received guidance and feedback from the project mentor.	complete

		onent		
17	16/4/2026	Final Hardw are Conne ctions	Final Connections are checked by mentor	completed